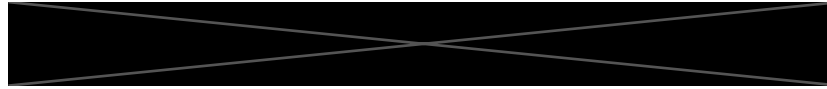


KT14603 DA [REDACTED] ALGORITHMS
PROJECT [REDACTED] OUP 14
Complaint Management System



1.0 Members and Contribution

No.	Name/Matric	Contribution
1	Syaza Syazana binti Anwar [REDACTED]	Main Programmer & Report Writer
2	Dorina Sharlene [REDACTED]	Menu Programmer & Report Writer
3	Norsylah binti Safrudin [REDACTED]	Main Programmer & Report Writer
4	Gloria binti Francis [REDACTED]	Data Structure Builder & Report Writer
5	Wan Nuradriana Balqis Binti Wan Azmi	Main tester code & Report Writer

2.0 System Specification and Data Structures

Following are the modules/functions in the system and the application of data structures.

1. Menus

- Complaint Storage and Retrieval Menu
 - This menu consists of 3 functions: create new complaints, view complaints and print complaints.
 - Create a new complaint function allows users to add new complaints. The menu then prompts the user to enter some details such as complaint number, name, email address, phone number, date of the complaint being issued and complaint description that is not more than 1000 words.

Add Your Complaint Details

Type Complaint no: 1

Enter Your Name: Syaza

Enter Your Email: [syazasy](#)
Enter Your Phone Number: 
Enter Date : 2/7/2023

Complaint Description: (1000 words maximum)
I have an issue regarding the service.

- View complaints function only accessible by the admin/staff. It allows the admin/staff to view complaints by category.

Enter the complaint category to view:

1. Hygiene
2. Service
3. Billing
4. Other

Enter the complaint category number: 1

```
=====
|          Complaint Management System          |
=====
Complaints in Category: Hygiene
-----
```

No complaints available.
Sorry, no complaint found in the selected category.

(this showed when there is no complaint available on selected category)

Enter the complaint category to view:

1. Hygiene
2. Service
3. Billing
4. Other

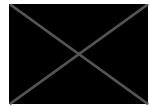
Enter the complaint category number: 2

```
=====
|          Complaint Management System          |
=====
Complaints in Category: Service
-----
```

Complaint Number: 1

Customer Name: syaza

Customer Email: [syazasya](#)

Customer Phone Number: 

Complaint Date: 2/7/2023

Complaint Description:

i have issue regarding the service.

Complaint Status: Not Solved

(this showed when there is complaint available on selected category)

- Print complaint function is only accessible by the admin/staff. It allows users to print the details of the complaints.

Enter the Complaint Number To Print The Complaint Details

1

Here is the Complaint list

Complaint Number : 1

Customer Name: Syaza

Customer Email: [syazasy](#)

Customer Phone Number

Order Date : 2/7/2023



Complaint description:

i have issue regarding the service.

Complaint Printed Successfully....!

- User Authentication and Access Control Menu

-Prompt the user to choose role(staff/customer)

```
=====
|      Complaint Management System      |
|      *****                        |
|      Hello! We are here to help you.  |v
=====
```

|Are you a: |

```
-----
|      1. Customer      |
|      2. Administrator/ Staff      |
|      3. Changed My Mind      |
-----
```

Please select your role:

-Output would be as above:

-After the user choose the role either staff/customer the following output will pop out:

Please enter your username:

Please enter your user password:

-the user will be asked to enter username and password

-the user will proceed to choose the complaint categorization

- Complaint Categorization Menu

- Presented users with a menu that displayed the categories of complaints to choose from. This will categorize the users' complaints based on the corresponding category number: (1) 'Hygiene', (2) 'Service', (3) 'Billing' and (default) 'Other'.
- Will take the user's input and assign the corresponding category string to the 'temp->category' member of the node structure.
- The code included error handling for invalid category choices. If a user enters a category number that is not listed (e.g, 4 or any other value), the default case would be triggered.

```
default:  
    category = "Other";  
    break;
```

- Complaint Resolution Menu

This module is used by employees who are in charge of handling complaints. It includes tools for developing and implementing action plans to address complaints. The module also allows staff to interact with complainants about the status of their complaints and track the implementation of these plans. In this module after the user key-in their complaint they can trace back the history for the complaint.

The `update_complaint()` function allows users to update the status of a complaint in the resolution module. The function takes one argument, the complaint number, and then searches the linked list of complaints for a complaint with that number. If a match is found, the function asks the user to provide a new description of the complaint and whether the complaint has been solved. The function then updates the complaint record with the new information.

```

else {
    while (temp != NULL) {
        if (temp->category == category) {
            cout << "Complaint Number: " << temp->complaint_number << endl;
            cout << "Customer Name: " << temp->customerName << endl;
            cout << "Customer Email: " << temp->customerEmail << endl;
            cout << "Customer Phone Number: " << temp->telNumber << endl;
            cout << "Complaint Date: " << temp->date << endl;
            cout << "Complaint Description:\n" << temp->x << endl;
            cout << "Complaint Status: " << (temp->solved ? "Solved" : "Not
Solved") << endl;

            cout <<
"-----\n";

            complaintFound = true;
        }
        temp = temp->next;
    }
}

```

- Complaint Report Menu

This Report menu allows only the Administrator/Staff to see through the Reporting of the complaints, only the Administrator or staff have the access to it with the set user name and user password. The complaint report consist of :

- The complaint category
- The percentage of complaints for the category
- The number of complaints for the category
- The highest number of complaints

As we Enter the 2nd Option it will show this output

```
=====
|          Complaint Management System          |
=====
|Administrator/Staff login                      |
=====
```

The user name :admin

The user password:admin

For the reporting menu, Administrator Enter option 5 which is the View Complaint Menu.

The Complaint Report will look like this

```
=====
|          Complaint Report          |
=====
Category          Percentage    Number of complaints
=====
Highest number of complaints:      (0 complaints)
```

2. Data Structures

- 'node' struct used to represent node in linked list data structure.
- Complaint (customerName, customerEmail, telNumber, date, x, category, solved).
- Customer complaints are represented by the complaint data structure.
- It contained attributes such as customer name, email, telephone number, date, category and a solved status.
- The complaints are stored within a list.

3. Linked List

- Linked list is used to store complaints as collections of nodes.
- Each node is containing the pointer to the next node in the list and the details of the complaints.
- To manipulate and traverse through the linked list, pointers are utilized.
- For example, the pointers are 'start_ptr', 'head', 'last', 'prev' and 'next'.

3.0 Screen Capture of the System

```
C:\Users\Honor\source\repos  x  +  v

=====
|                               |
|      Complaint Management System      |
|*****|
|      Hello! We are here to help you.  |
|                               |
|-----|
|Are you a:                            |
|-----|
|      1. Customer                      |
|      2. Administrator/ Staff          |
|      3. Changed My Mind                |
|-----|
|
|Please select your role: 1

=====
```

```
C:\Users\Honor\source\repos  x  +  v

=====
|                               |
|      Complaint Management System      |
|-----|

Add Your Complaint Details
-----
Type Complaint no: 2

Enter Your Name: sylah
Enter Your Email: syeoll!
Enter Your Phone Number:
Enter Date : 6/7/2023

Complaint Description: ( 1000 words maximum )
sghgdhsdgsd

What is your complain category?
1. Hygiene
2. Service
3. Billing
Enter category number: 1
=====
|                               |
|      Complaint added Successfully      |
|-----|
Press any key to continue . . . |
```

```
C:\Users\Honor\source\repos  X  +  v

=====
|      Complaint Management System      |
|      *****                        |
|      Hello! We are here to help you. |
|=====|

-----
|Are you a:                             |
|-----|
|      1. Customer                      |
|      2. Administrator/ Staff         |
|      3. Changed My Mind              |
|-----|

Please select your role: 2|
```

```
C:\Users\Honor\source\repos  X  +  v

=====
|      Complaint Management System      |
|=====|

-----
|      Administrator/ Staff Login      |
|-----|

Please enter your username: admin

Please enter your user password: admin|
```

```
C:\Users\Honor\source\repos  X  +  v  -  0  X

=====
|      Complaint Management System      |
|=====|

-----
|      1. Create Complaint              |
|-----|
|      2. View Complaint               |
|-----|
|      3. Update Complaint              |
|-----|
|      4. Print Complaint               |
|-----|
|      5. View Complaint Report         |
|-----|
|      6. Log Out                      |
|-----|

Enter choice:
```

```
C:\Users\Honor\source\repos  X  +  v

=====
|                               |
|          Complaint Report    |
|                               |
=====

Category      Percentage      Number of complaints
-----
Service       50.00%          1
Hygiene       50.00%          1

Highest number of complaints: Service (1 complaints)

Press any key to continue . . .
```

```
C:\Users\Honor\source\repos  X  +  v

Enter the complaint category to view:
1. Hygiene
2. Service
3. Billing
4. Other
Enter the complaint category number: 2

=====
|                               |
|          Complaint Management System    |
|                               |
=====

Complaints in Category: Service
-----
Complaint Number: 2
Customer Name: sylah
Customer Email: syeol19
Customer Phone Number:
Complaint Date: 6/7/2023
Complaint Description:
sghggdhgs
Complaint Status: Solved
-----

Press any key to continue . . . |
```

```
C:\Users\Honor\source\repos  X  +  v

|          3. Update Complaint          |
|-----|
|          4. Print Complaint           |
|-----|
|          5. View Complaint Report     |
|-----|
|          6. Log Out                  |
|-----|

Enter choice: 6
You are Logged Out...!

Press any key to continue . . .

You choose to exit.

thank you
Press any key to continue . . .

=====
|                               |
|          Complaint Management System    |
|          *****              |
|          Hello! We are here to help you. |
|                               |
=====

|Are you a: |
|-----|
|          1. Customer              |
|          2. Administrator/ Staff  |
|          3. Changed My Mind       |
|-----|

Please select your role:
```

4.0 Code Listing

```
#include <iostream>
#include <stdlib.h>
#include <string>
#include <cctype>
#include <fstream>
#include <iomanip>

using namespace std;

class complaint
{
public:
    void customer_StoreComplaint();
    void admin_menu();
    void create_complaint();
    void update_complaint();
    void print_complaint();
    void view_complaint();
    void monthly_complaint_report();
    void exit();

    complaint();
};

struct node
{
    int complaint_number; //Complain number is to identify each complaints
    //Customer complaint number is impossible to be the same
    string customerName;
    string customerEmail;
    string telNumber;
    string date;
    string x;
    string category;
    bool solved;

    node* prev;
    node* next;
    node* link;
} *q, * temp;

node* start_ptr = NULL;
node* head = NULL;
node* last = NULL;
```

```

// Constructor definition
complaint::complaint()
{
    /* Even if the constructor implementation is empty,
       such as fulfilling the requirement of having a constructor
       defined for the class.
       Even if the constructor doesn't really do anything,
       it is still required by the class definition.
    */
}

int main()
{
    complaint admin;
    int option;
    do
    {
        cout << "\n";
        cout << "
=====
        cout << "|      Complaint Management System      |\n";
        cout << "|      *****                          |\n";
        cout << "|      Hello! We are here to help you.    |\n";
        cout << "
=====
        cout << "-----\n";
        cout << "| Are you a: \t\t\t\t|\n";
        cout << "-----\n";
        cout << "| \t1. Customer \t\t\t\t|\n";
        cout << "| \t2. Administrator/ Staff \t\t|\n";
        cout << "| \t3. Changed My Mind \t\t\t|\n";
        cout << "-----\n\n";

        cout << "Please select your role: ";
        cin >> option;
        switch (option)
        {
            case 1:
                admin.customer_StoreComplaint();
                break;
            case 2:
                admin.admin_menu();
                break;
            case 3:
                admin.exit();
                goto a;
                break;
        }
    } while (option != 0);
}

```

```

    }

    } while (option != 3);
a:
    cout << "    ^ ^ \n";
    cout << " ( o.o )\n";
    cout << " > ^ < \n";
    cout << " Thank you for your time.\n\n" << endl;
    system("PAUSE");

}

void complaint::customer_StoreComplaint()
{
    // This function allows customers to add their complaints and details

    system("cls"); // Clear the console screen
    cout << "\n";
    cout << "
===== \n";
    cout << "|          Complaint Management System          | \n";
    cout << "
===== \n\n";

    cout << "\nAdd Your Complaint Details\n";
    cout << " _____ \n\n";

    // Create a new node to store the complaint details
    node* temp;
    temp = new node;

    //Generate a unique complaint number
    bool isUnique = false;
    int complaintNumber;
    while (!isUnique) {
        cout << "Type Complaint no: ";
        cin >> complaintNumber;

        //Check if the complaint number already exists
        node* ptr = start_ptr;
        isUnique = true;
        while (ptr != NULL) {
            if (ptr->complaint_number == complaintNumber) {
                isUnique = false;
                cout << "Complaint number already exists. Please choose a
different number.\n";
                break;
            }
            ptr = ptr->next;
        }
    }
}

```

```

    }
}
temp->complaint_number = complaintNumber;

//This code here ask customer to input their details
cout << "\nEnter Your Name: ";
cin.ignore(); //
getline(cin, temp->customerName);
cout << "Enter Your Email: ";
getline(cin, temp->customerEmail);
cout << "Enter Your Phone Number: ";
getline(cin, temp->telNumber);
cout << "Enter Date : ";
cin >> temp->date;
cout << "\nComplaint Description:";
cout << " ( 1000 words maximum ) \n";
cin.ignore();
getline(cin, temp->x);

/*This is the categorization module.
Complaints can be categorized into 4 categories:
Hygiene, Service, Billing and Other
*/
cout << endl;
cout << "What is your complain category?\n";
cout << "1. Hygiene\n";
cout << "2. Service\n";
cout << "3. Billing\n";
cout << "Enter category number: ";

int categoryChoice;
cin >> categoryChoice;

/*Variable categoryChoice declared
Its value will be assigned by the
the data type is unsigned int so the value assigned will only be positive numbers
*/
cin.ignore();

/*
This switch statement will assign value to the category (a variable)
based on the value of categoryChoice (1,2,3 and any other value)
*/
string category;
switch (categoryChoice) {
case 1:
    category = "Hygiene";
    break;

```

```

        case 2:
            category = "Service";
            break;
        case 3:
            category = "Billing";
            break;
        default:
            category = "Other";
            break;
    }

    temp->category = category;

    cout <<
    "=====
    =====>" << endl;
    cout << "Complaint added Successfully" << endl;
    cout <<
    "=====
    =====>" << endl;

    system("PAUSE");

    //This code here used linked list
    //It demonstrates the process of inserting a new node at the beginning of a linked list
    temp->next = NULL;
    if (start_ptr != NULL)
    {
        temp->next = start_ptr;
    }
    start_ptr = temp;
    system("cls");
}

void complaint::admin_menu()
{
    complaint admin;
    int menu;
    string userName, userPassword;
    //user registration
    //User need to choose either staff/administrator or custpmer
    //Customer/staff will immediately prompt to the complaint interface
    system("cls");
    cout << "\n";
    cout << "
    =====>\n";

```

```

        cout << "|          Complaint Management System          |\n";
        cout << "
=====\\n\\n";
        cout << " -----\\n";
        cout << "|          Administrator/ Staff Login          |\n";
        cout << " -----\\n\\n";
        cout << "\\nPlease enter your username: ";
        cin >> userName;
        cout << "\\nPlease enter your user password: ";
        cin >> userPassword;

        if (userName == "admin" && userPassword == "admin") {
            do
            {
                system("cls");
                cout << "\\n";
                cout << "
=====\\n";
                cout << "|          Complaint Management System          |\n";
                cout << "
=====\\n\\n";
                cout << " -----\\n";
                cout << "|\\t1. Create Complaint \\t\\t\\t\\t\\t\\n";
                cout << " -----\\n";
                cout << " -----\\n";
                cout << "|\\t2. View Complaint \\t\\t\\t\\t\\t\\n";
                cout << " -----\\n";
                cout << " -----\\n";
                cout << "|\\t3. Update Complaint \\t\\t\\t\\t\\t\\n";
                cout << " -----\\n";
                cout << " -----\\n";
                cout << "|\\t4. Print Complaint \\t\\t\\t\\t\\t\\n";
                cout << " -----\\n";
                cout << " -----\\n";
                cout << "|\\t5. View Complaint Report  \\t\\t\\t\\t\\t\\n";
                cout << " -----\\n";
                cout << " -----\\n";
                cout << "|\\t6. Log Out \\t\\t\\t\\t\\t\\t\\n";
                cout << " -----\\n\\n";

                cout << "Enter choice: ";

                cin >> menu;

                switch (menu)
                {
                    case 1:
                    {

```

```

        admin.create_complaint();
        break;
    }

    case 2:
    {
        admin.view_complaint();
        system("PAUSE");
        break;
    }

    case 3:
    {
        admin.update_complaint();
        system("PAUSE");
        break;
    }

    case 4:
    {
        admin.print_complaint();
        system("PAUSE");
        break;
    }

    case 5:
    {
        admin.monthly_complaint_report();
        system("PAUSE");
        break;
    }

    case 6:
    {
        cout << "You are Logged Out...\n\n\n";
        system("PAUSE");
        admin.exit();
        break;
    }

    }//end Switch

} while (menu != 6);//end do
cout << "thank you" << endl;
system("PAUSE");
}

```

```

else
{
    cout << "\n\n\t\tInvalid login attempt. Please try again.\n" << "\n";
    system("PAUSE");
    system("cls");
    admin.admin_menu();
}
}

void complaint::create_complaint()
{
    system("cls");

    //This create_complaint function is to allow staff/admin to create complains for customer
    //This part mainly controlled by staff/admin who have access to the system
    cout << "\n";
    cout << "\t\t
===== \n";
    cout << "\t\t\t\t\tComplaint Management System\t\t\t\t\t| \n";
    cout << "\t\t
===== \n\n";
    cout << "\nAdd Complaint Details of Customer\n";
    cout << "_____ \n\n";

    /*This part creates new node for a linked list, prompt user to enter various details
    Utilizes dynamic memory allocation (new) to create the node and getline to handle input
with spaces*/
    node* temp;
    temp = new node;
    cout << "Type Complaint no: ";
    cin >> temp->complaint_number;
    cout << "Enter Customer Name: ";
    cin.ignore();
    getline(cin, temp->customerName);
    cout << "Enter Your Email: ";
    getline(cin, temp->customerEmail);
    cout << "Enter Your Phone Number: ";
    getline(cin, temp->telNumber);
    cout << "Enter Date : ";
    cin >> temp->date;
    cout << "\nComplaint Description:";
    cout << "( 1000 words maximum ) \n";
    cin.ignore();
    getline(cin, temp->x);

```

```

/*This is the categorization module.
Complaints can be categorized into 4 categories:
Hygiene, Service, Billing and Other
*/
cout << "Please enter complain category.:\n";
cout << "1. Hygiene\n";
cout << "2. Service\n";
cout << "3. Billing\n";
cout << "Enter category number: ";

int categoryChoice;
cin >> categoryChoice;

/*Variable categoryChoice declared
Its value will be assigned by the
the data type is unsigned int so the value assigned will only be positive numbers
*/
cin.ignore();

/*
This switch statement will assign value to the category (a variable)
based on the value of categoryChoice (1,2,3 and any other value)
*/
string category;
switch (categoryChoice) {
case 1:
    category = "Hygiene";
    break;
case 2:
    category = "Service";
    break;
case 3:
    category = "Billing";
    break;
default:
    category = "Other";
    break;
}

temp->category = category;

cout <<
"=====
===== " << endl;
cout << "Complaint added Successfully" << endl;

```

```

        cout <<
"=====
===== " << endl;
        cout << "Go to Print Complaint to print the Complaint" << endl;
        cout <<
"=====
===== " << endl;
        system("PAUSE");

        temp->next = NULL;
        if (start_ptr != NULL)
        {
            temp->next = start_ptr;
        }
        start_ptr = temp;
        system("cls");
    }

```

```

void complaint::view_complaint() {
    system("cls");
    //Admin can view complaints based on their categories.
    //There are 4 types of categories that admin can choose from: Hygiene, Service, Billing,
Other
    cout << "Enter the complaint category to view:\n";
    cout << "1. Hygiene\n";
    cout << "2. Service\n";
    cout << "3. Billing\n";
    cout << "4. Other\n";
    cout << "Enter the complaint category number: ";

    int categoryChoice;
    //This line will get input from the admin
    cin >> categoryChoice;

    //This will tell the program to ignore the newline character within the input
    cin.ignore();

    string category;
    switch (categoryChoice) {
        case 1:
            category = "Hygiene";
            break;
        case 2:
            category = "Service";
            break;
        case 3:

```

```

        category = "Billing";
        break;
default:
    category = "Other";
    break;
}

// Code to view complaints by category
bool complaintFound = false;
node* temp = start_ptr;

cout << "\n";
cout << "
=====
cout << "|          Complaint Management System          |\n";
cout << "
=====
cout << "Complaints in Category: " << category << "\n";
cout << "-----\n";

if (temp == NULL) {
    cout << "No complaints available.\n";
}
else {
    while (temp != NULL) {
        if (temp->category == category) {
            cout << "Complaint Number: " << temp->complaint_number <<
endl;

            cout << "Customer Name: " << temp->customerName << endl;
            cout << "Customer Email: " << temp->customerEmail << endl;
            cout << "Customer Phone Number: " << temp->telNumber <<
endl;

            cout << "Complaint Date: " << temp->date << endl;
            cout << "Complaint Description:\n" << temp->x << endl;
            cout << "Complaint Status: " << (temp->solved ? "Solved" : "Not
Solved") << endl;

            cout <<
"-----\n";

            complaintFound = true;
        }
        temp = temp->next;
    }
}

if (!complaintFound) {
    cout << "Sorry we No complaints found in the selected category.\n";
}

```

```

        system("PAUSE");
    }

```

// This is Resolution Complain Module

/* The system("cls") command clears the console screen to provide a clean interface for the user.

The variable sid is declared to store the complaint number entered by the user, and the boolean variable found is initialized as false.

The variable temp is set to point to the start of the complaint list (assuming it is a linked list implementation). */

```

void complaint::update_complaint() {
    system("cls");
    int sid = 0;
    //This is to initialize sid with a default value first
    bool found;
    found = false;
    temp = start_ptr;
    cout << "\n";
    cout << "
=====
    cout << "|          Complaint Management System          |\n";
    cout << "
=====
    cout << "\n\n\n";
    cout << "Enter Complaint Number To Update: ";
    cin >> sid;

```

/* If temp is NULL and sid is 0, it means there are no complaints in the system, so a message is displayed to inform the user.

If there are complaints in the system, a while loop is executed as long as temp is not NULL and found is false.

Inside the while loop, the program checks if the complaint number stored in temp->complaint_number matches the sid entered by the user.

If a match is found, the found variable is set to true, and the loop terminates. Otherwise, temp is updated to point to the next complaint in the list. */

```

    if (temp == NULL && sid == 0) {
        cout << "No Complaint To Update..!" << endl;
    }
    else {
        while (temp != NULL && !found) {
            if (temp->complaint_number == sid) {
                found = true;
            }

```

```

        else {
            temp = temp->next;
        }
    }
    if (found) {
        cout << "\nCustomer Name: " << temp->customerName;
        cout << "\nCustomer Email: " << temp->customerEmail;
        cout << "\nCustomer Phone Number: " << temp->telNumber << endl;
        cout << endl;
        cout << "\nExplain Solution of the Complaint:" << endl;
        cout << "( 1000 words maximum ) \n";
        cout << " ";
        cin.ignore();
        getline(cin, temp->x);
        cout << endl;
        cout << "Is the complaint solved? \n(1 for yes, 0 for no): ";
        cout << endl;
        cin >> temp->solved; /* The program asks the user whether the
complaint has been solved or not,
                                expecting a 1 for
yes and 0 for no.
                                The input is stored in
temp->solved. */

                                /*
                                This is the
categorization module
                                Complaints can be
categorized into 4 categories: Hygiene, Service, Billing and Other
                                */

        cout << "Please enter complain category:\n";
        cout << "1. Hygiene\n";
        cout << "2. Service\n";
        cout << "3. Billing\n";
        cout << "Enter category number: ";

        /*Variable categoryChoice declared
        Its value will be assigned by the
        the data type is unsigned int so the value assigned will only be positive
numbers
        */
        unsigned int categoryChoice;
        cin >> categoryChoice;

        // This is to discard the newline character
        //To avoid confusion caused by leftover characters in the input
        cin.ignore();

```

```

        string category;
        /*
        This switch statement will assign value to the category (a variable)
        based on the value of categoryChoice (1,2,3 and any other value)
        */
        switch (categoryChoice) {
        case 1:
            category = "Hygiene";
            break;
        case 2:
            category = "Service";
            break;
        case 3:
            category = "Billing";
            break;
        default:
            category = "Other";
            break;
        }

        temp->category = category;

        system("PAUSE");
        temp = temp->next;
    }
    cout << "COMPLAINT RECORD UPDATED....!\n\n" << endl;
}
}

```

```

void complaint::monthly_complaint_report() {
    system("cls");

    // Define an array to store the complaint categories and their corresponding counts
    const int MAX_CATEGORIES = 10;
    string complaint_categories[MAX_CATEGORIES];
    int complaint_counts[MAX_CATEGORIES] = { 0 };

    // Iterate through the linked list of complaints
    node* temp = start_ptr;
    while (temp != NULL) {
        // Get the complaint category
        string category = temp->category;

        // Find the category in the array
        int categoryIndex = -1;
        for (int i = 0; i < MAX_CATEGORIES; i++) {
            if (complaint_categories[i] == category) {
                categoryIndex = i;
            }
        }
    }
}

```

```

        break;
    }
}

if (categoryIndex != -1) {
    // Increment the count of the existing complaint category
    complaint_counts[categoryIndex]++;
}
else {
    // Add a new complaint category to the array
    for (int i = 0; i < MAX_CATEGORIES; i++) {
        if (complaint_categories[i].empty()) {
            complaint_categories[i] = category;
            complaint_counts[i] = 1;
            break;
        }
    }
}

temp = temp->next;
}

// Calculate the percentage of complaints for each category
int total_complaints = 0;
for (int i = 0; i < MAX_CATEGORIES; i++) {
    total_complaints += complaint_counts[i];
}

// Find the highest number of complaints
float highest_number_of_complaints = 0;
string highest_category = "";
for (int i = 0; i < MAX_CATEGORIES; i++) {
    if (complaint_counts[i] > highest_number_of_complaints) {
        highest_number_of_complaints = complaint_counts[i];
        highest_category = complaint_categories[i];
    }
}

// Delete complaints if the total number of complaints exceeds 100
if (total_complaints > 100) {
    int excess_complaints = total_complaints - 100;
    node* current = start_ptr;
    node* previous = NULL;
    int complaints_deleted = 0;

    while (current != NULL && complaints_deleted < excess_complaints) {
        node* next = current->next;
        delete current;
    }
}

```

```

        current = next;
        complaints_deleted++;
    }

    if (previous != NULL) {
        previous->next = current;
    }
    else {
        start_ptr = current;
    }
}

// Print the monthly complaint report
cout << "\n";
cout << "
=====
=====\\n";
    cout << "|                Complaint Report                | \\n";
    cout << "
=====
=====\\n\\n";
    //print column headers
    cout << left << setw(20) << " Category";
    cout << setw(12) << "Percentage";
    cout << setw(25) << "    Number of complaints " << "\\n";
    //print separating line made of dashes
    cout << setfill('-') << setw(63) << "" << "\\n";
    cout << setfill(' ');

    //Iterate through each category
    for (int i = 0; i < MAX_CATEGORIES; i++) {
        if (!complaint_categories[i].empty()) {
            //calculate the percentage as a float with 2 decimal places
            float percentage = (complaint_counts[i] /
static_cast<float>(total_complaints)) * 100;
            //print category,percentage(formatted with 2 decimal places)
            //add number of complaints in a neatly aligned manner
            cout << left << setw(20) << complaint_categories[i];
            cout << right << fixed << setprecision(2) << setw(7) << percentage
<< "%";
            cout << right << setw(20) << complaint_counts[i] << "\\n";
        }
    }

    cout << "\\n";
    //print the category with the highest number of complaints
    cout << "Highest number of complaints: " << highest_category << " (" << fixed <<
setprecision(0) << highest_number_of_complaints << " complaints)\\n";

```

```

        cout << endl;
    }

void complaint::print_complaint()
{
    int num;
    bool found;
    system("cls");

    /*This code here creates an 'ofstream' object to handle file output,
    opens a file named "prototype.txt" in write mode,
    and declares a pointer variable of type 'node'*/
    ofstream outputFile;
    outputFile.open("prototype.txt");
    node* temp;

    temp = start_ptr;
    found = false;
    cout << "\n";
    cout << "
=====
    cout << "|          Complaint Management System          |\n";
    cout << "
=====
    outputFile << "\n";
    outputFile << "
=====
    outputFile << "|          Complaint Management System          |\n";
    outputFile << "
=====
    cout << " Enter the Complaint Number To Print The Complaint Details\n";
    cin >> num;
    cout << "\n";
    cout << "-----" << endl;
    cout << "          Here is the Complaint list\n";
    cout << "-----" << endl;

    if (temp == NULL)
    {
        cout << "There is no Complaint to show\nSo The List is Empty\n\n\n";
    }
    while (temp != NULL && !found)
    {
        if (temp->complaint_number == num)
        {
            found = true;

```

```

    }
    else
    {
        temp = temp->next;
    }
    if (found)
    {
        cout << "Complaint Number : " << temp->complaint_number;
        outputFile << "Complaint Number : " << temp->complaint_number <<
endl;

        cout << "\n";
        cout << "Customer Name: " << temp->customerName << endl;
        outputFile << "Customer Name: " << temp->customerName << endl;
        cout << "Customer Email: " << temp->customerEmail << endl;
        outputFile << "Customer Email: " << temp->customerEmail << endl;
        cout << "Customer Phone Number: " << temp->telNumber << endl;
        outputFile << "Customer Phone Number: " << temp->telNumber <<
endl;

        cout << "Order Date : " << temp->date << endl;
        outputFile << "Order Date : " << temp->date << endl;
        cout <<
"
<< endl;
        outputFile <<
"
<< endl;

        cout << "Complaint description: " << endl;
        outputFile << "Complaint description: " << endl;
        cout << "-----" << endl;
        outputFile <<
"-----" << endl;

        cout << temp->x;
        outputFile << temp->x << endl;
        cout << "\n";
        cout <<
"
<< endl;
        outputFile <<
"
<< endl;

        temp = temp->next;
    }
    outputFile.close();
    cout << "Complaint Printed Succesfully....!\n";
}

```

```
}
```

```
void complaint::exit()
```

```
{
```

```
    cout << "\nYou choose to exit.\n" << endl;
```

```
}
```